Лабораторная работа №1
Сортировка и поиск

Цель работы

Реализовать сортировки и поиск на C++ и сравнить время работы с теоретической сложностью.

Порядок измерений

Использовались сортировки вставками, выбором и слиянием, а также линейный и бинарный поиск. Для каждой точки выполнено 10 повторов с усреднением. Каждый повтор включал не менее 100 мс измеряемого времени. Подготовка массивов и проверка результатов не учитывались.

На графиках показано среднее время одной операции в миллисекундах. Обе оси имеют логарифмический масштаб.

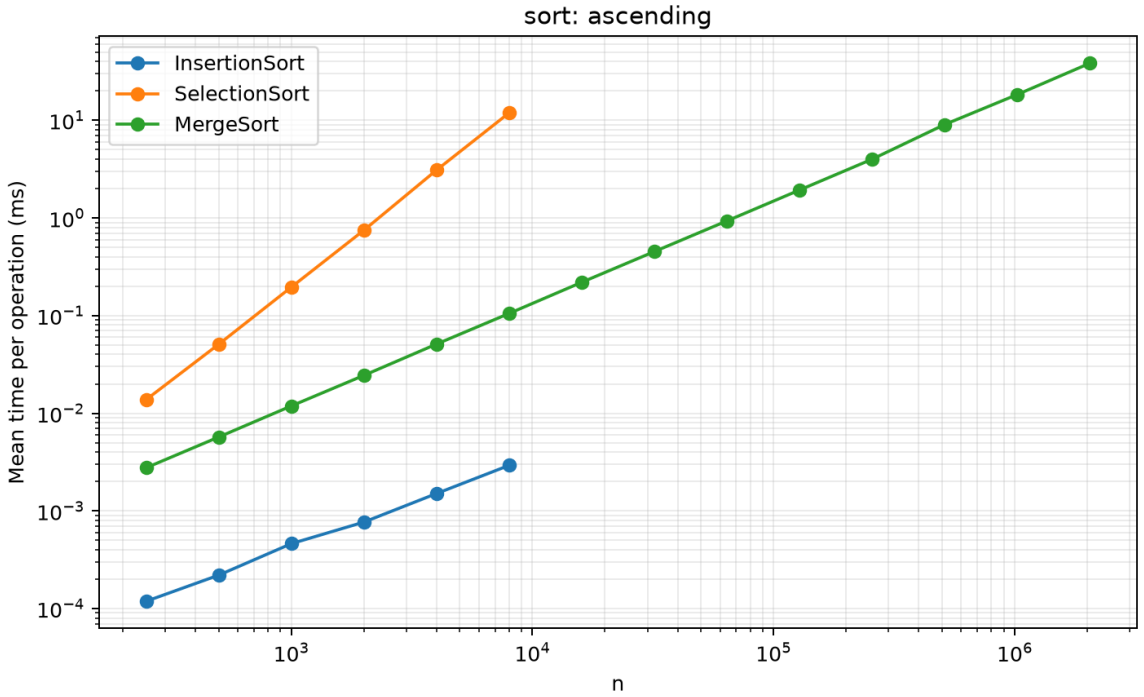


Рисунок 1. Сортировка возрастающего массива.

На возрастающем массиве вставки работают быстрее остальных сортировок. Сдвигать элементы не нужно, поэтому время растёт примерно линейно, O(n). Выбор сохраняет $O(n^2)$. Слияние имеет сложность O(n log n).

Среднее время сортировки при n = 8000, мс

Массив	Вставки	Выбор	Слияние
Возрастающий	0,00292	11,90	0,105
Убывающий	10,71	12,32	0,105
Случайный	5,51	12,42	0,456

Сортировка убывающего и случайного массива

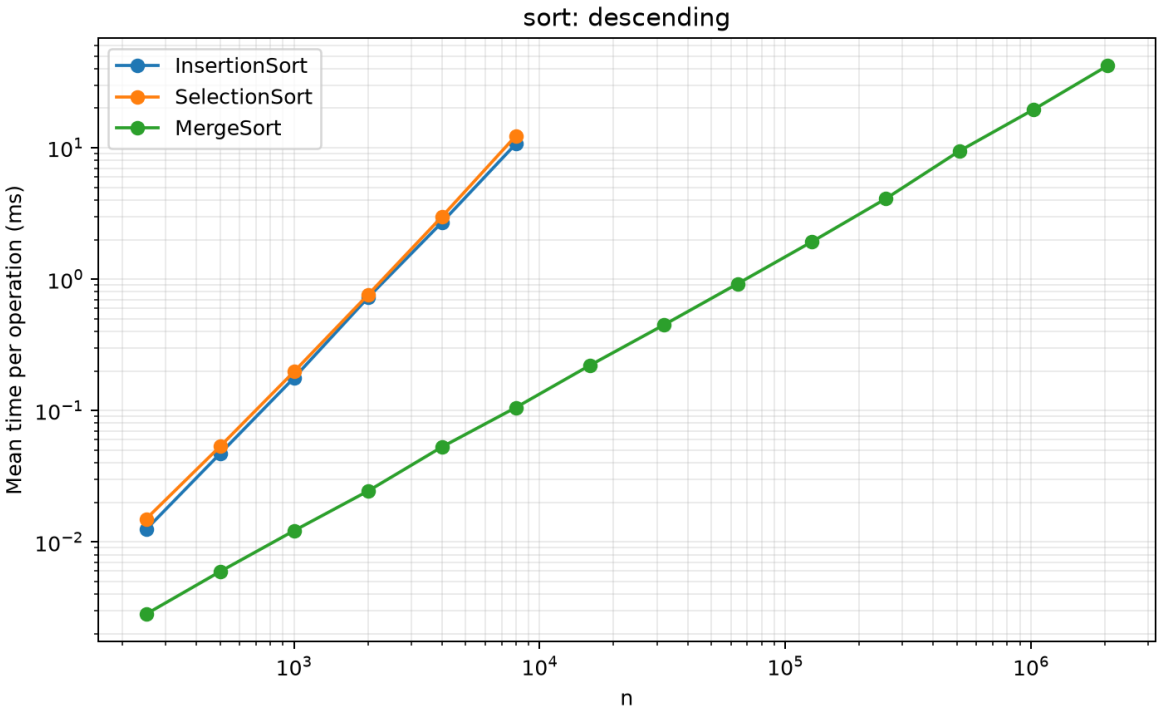


Рисунок 2. Сортировка убывающего массива.

Для вставок это худший случай. Каждый новый элемент приходится перемещать в начало. При росте n с 4000 до 8000 время увеличилось с 2,70 до 10,71 мс, почти в четыре раза. Это соответствует $O(n^2)$. Выбор показывает похожий рост.

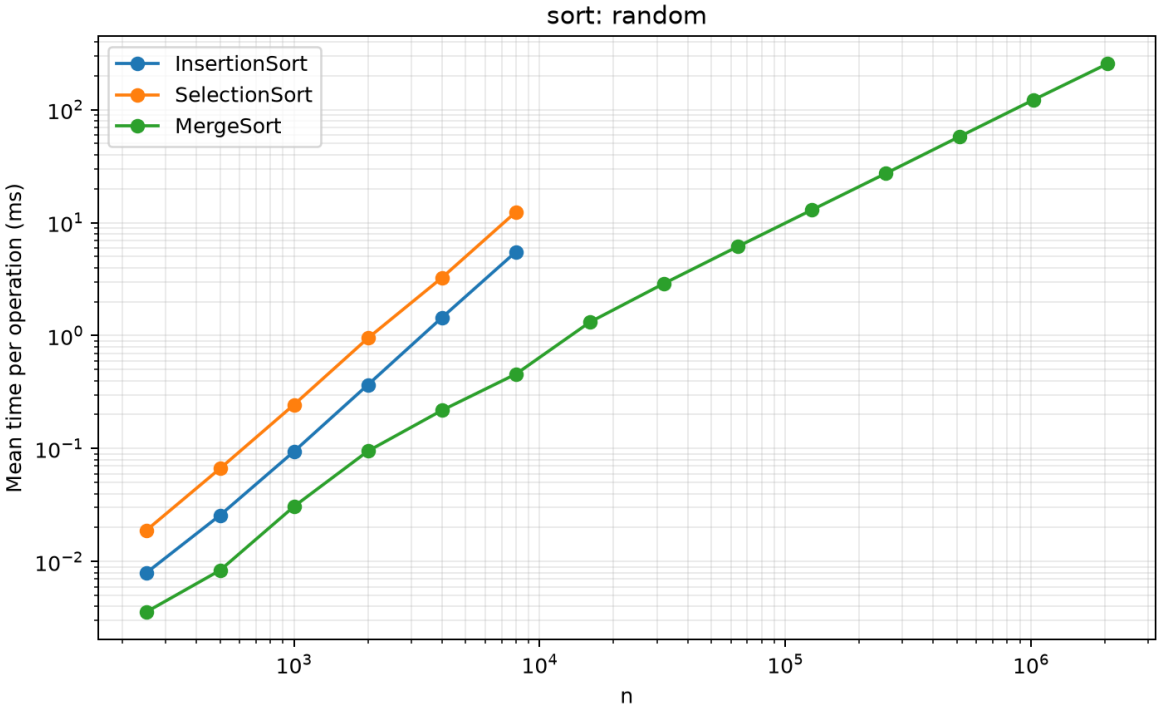


Рисунок 3. Сортировка случайного массива.

На случайном массиве вставки и выбор также показывают рост около $O(n^2)$. При n = 8000 слияние быстрее вставок примерно в 12 раз, а выбора в 27 раз. На больших n удвоение размера увеличивает время слияния чуть больше чем вдвое. Это согласуется с O(n log n).

Поиск

Поиск выполнялся в упорядоченном массиве. Для успешного поиска перебирались все его значения. Отдельно выполнялись запросы отсутствующего значения. Время делилось на число запросов.

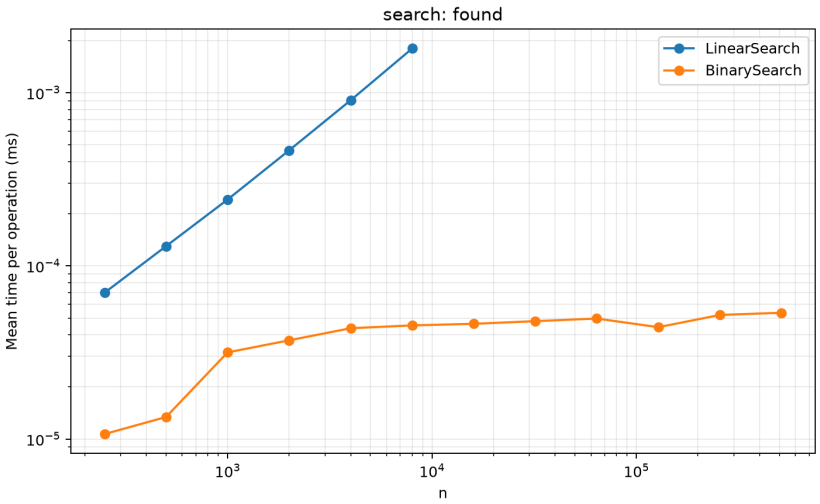


Рисунок 4. Поиск существующего элемента.

Линейный поиск в среднем проверяет около половины массива. Время растёт примерно как O(n). Бинарный поиск делит диапазон пополам и имеет сложность O(log n). При n = 8000 он быстрее примерно в 40 раз.

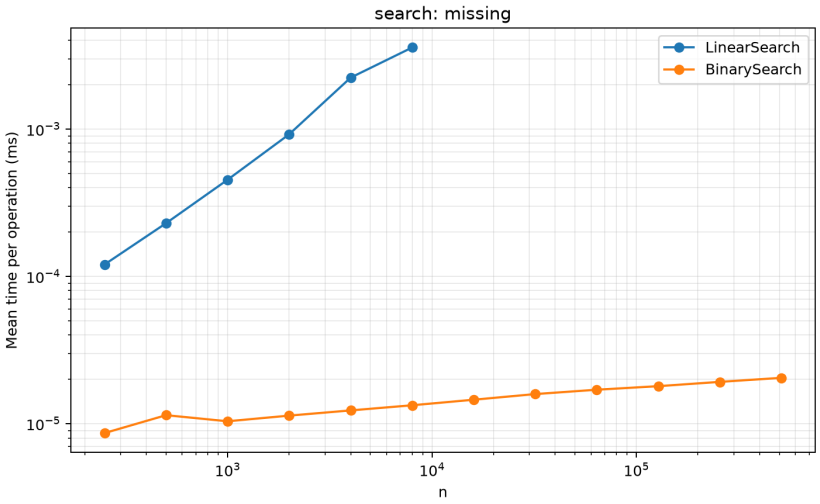


Рисунок 5. Поиск отсутствующего элемента.

Если элемента нет, линейный поиск просматривает весь массив. При n = 8000 это занимает примерно вдвое больше времени. Сложность остаётся O(n). Для бинарного поиска сохраняется O(log n).

Запрос при n = 8000	Линейный, нс	Бинарный, нс
Элемент присутствует	1799	45,3
Элемент отсутствует	3579	13,3

Вывод

Результаты согласуются с теорией. Вставки подходят для уже упорядоченного массива. Слияние быстрее на больших неупорядоченных массивах. В упорядоченном массиве бинарный поиск значительно быстрее линейного.

Приложение. Код программы

```
#include <algorithm>
#include <chrono>
#include <climits>
#include <cmath>
#include <cstdint>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <iterator>
#include <numeric>
#include <random>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

using namespace std;
using namespace std::chrono;
using T = int;
using Vector = vector<T>;
using SortFunction = void (*)(Vector& v);
using SearchFunction = int (*)(Vector& v, const T& key);
enum Order { Asc, Dsc, Rnd };
using Clock = steady_clock;

// ================================================
// Вспомогательные функции из шаблона
// ================================================
istream& operator>>(istream& is, Vector& v) {
    v.clear();
    string s;
    getline(is, s);
    istringstream ss(s);
    copy(istream_iterator<T>(ss), istream_iterator<T>(), back_inserter(v));
    return is;
}
ostream& operator<<(ostream& os, const Vector& v) {
    copy(v.begin(), v.end(), ostream_iterator<T>(os, " "));
    return os;
}
Vector AscendingVector(size_t n) {
    Vector v(n);
    iota(v.begin(), v.end(), 0);
    return v;
}
Vector DescendingVector(size_t n) {
    Vector v(n);
    iota(v.rbegin(), v.rend(), 0);
    return v;
}
mt19937 generator(42); // Фиксированный seed для воспроизводимости.
Vector RandomVector(size_t n, T a = 0, T b = 1'000'000) {
    if (a > b) throw invalid_argument("RandomVector: a > b");
    Vector v(n);
    uniform_int_distribution<T> distribution(a, b);
    for (T& value : v) value = distribution(generator);
    return v;
}
Vector MakeVector(Order order, size_t n) {
    if (order == Asc) return AscendingVector(n);
    if (order == Dsc) return DescendingVector(n);
    return RandomVector(n);
}

// ================================================
// Эталонные функции преподавателя: только проверки!
// В экспериментальных сериях они не участвуют.
// ================================================
void StdSort(Vector& v) { sort(v.begin(), v.end()); }
int StdLinearSearch(Vector& v, const T& key) {
    const auto it = find(v.begin(), v.end(), key);
    return it == v.end() ? -1 : static_cast<int>(it - v.begin());
```

Код программы (продолжение)

```
}
int StdBinarySearch(Vector& v, const T& key) {
    const auto it = lower_bound(v.begin(), v.end(), key);
    return it == v.end() || *it != key ? -1 : static_cast<int>(it - v.begin());
}

// ================================================
// Собственные алгоритмы сортировки
// ================================================
void InsertionSort(Vector& v) {
    for (size_t i = 1; i < v.size(); ++i) {
        const T value = v[i];
        size_t j = i;
        while (j > 0 && v[j - 1] > value) {
            v[j] = v[j - 1];
            --j;
        }
        v[j] = value;
    }
}
void SelectionSort(Vector& v) {
    for (size_t i = 0; i < v.size(); ++i) {
        size_t smallest = i;
        for (size_t j = i + 1; j < v.size(); ++j)
            if (v[j] < v[smallest]) smallest = j;
        if (smallest != i) swap(v[i], v[smallest]);
    }
}
// Полуинтервал [left, right): индекс right не входит в диапазон.
void MergeRange(Vector& v, Vector& buffer, size_t left, size_t right) {
    if (right - left < 2) return;
    const size_t middle = left + (right - left) / 2;
    MergeRange(v, buffer, left, middle);
    MergeRange(v, buffer, middle, right);
    size_t i = left, j = middle, k = left;
    while (i < middle && j < right) {
        if (v[i] <= v[j]) buffer[k++] = v[i++];
        else buffer[k++] = v[j++];
    }
    while (i < middle) buffer[k++] = v[i++];
    while (j < right) buffer[k++] = v[j++];
    for (k = left; k < right; ++k) v[k] = buffer[k];
}
void MergeSort(Vector& v) {
    Vector buffer(v.size());
    MergeRange(v, buffer, 0, v.size());
}

// ================================================
// Собственные алгоритмы поиска
// Возвращают индекс или -1, как в шаблоне.
// ================================================
int LinearSearch(Vector& v, const T& key) {
    for (size_t i = 0; i < v.size(); ++i)
        if (v[i] == key) return static_cast<int>(i);
    return -1;
}
int BinarySearch(Vector& v, const T& key) {
    // Предусловие: v упорядочен по неубыванию.
    // Находим первое вхождение, как StdBinarySearch в шаблоне.
    size_t left = 0, right = v.size();
    while (left < right) {
        const size_t middle = left + (right - left) / 2;
        if (v[middle] < key) left = middle + 1;
        else right = middle;
    }
    return left < v.size() && v[left] == key ? static_cast<int>(left) : -1;
}

// ================================================
// Проверки корректности; работают и в release-сборке.
// ================================================
```

Код программы (продолжение)

```
void Require(bool condition) {
    if (!condition) throw runtime_error("Algorithm correctness check failed");
}
void CheckVector(const Vector& initial) {
    Vector expected = initial;
    StdSort(expected);
    for (SortFunction func : {InsertionSort, SelectionSort, MergeSort}) {
        Vector v = initial;
        func(v);
        Require(v == expected); // Проверка порядка И сохранения элементов.
    }
    Vector v = expected;
    for (T key : v) {
        Require(LinearSearch(v, key) == StdLinearSearch(v, key));
        Require(BinarySearch(v, key) == StdBinarySearch(v, key));
    }
    for (T key : {INT_MIN, -1000, -2, 0, 2, 1000, INT_MAX}) {
        Require(LinearSearch(v, key) == StdLinearSearch(v, key));
        Require(BinarySearch(v, key) == StdBinarySearch(v, key));
    }
}
void CheckAlgorithms() {
    // Все массивы длины 0..7 над {-1, 0, 1}, в том числе дубли.
    for (int n = 0, combinations = 1; n <= 7; ++n, combinations *= 3) {
        for (int code = 0; code < combinations; ++code) {
            Vector v;
            for (int i = 0, x = code; i < n; ++i, x /= 3) v.push_back(x % 3 - 1);
            CheckVector(v);
        }
    }
    CheckVector({INT_MAX, 0, INT_MIN, INT_MAX});
    for (int i = 0; i < 20; ++i) CheckVector(RandomVector(100, -100, 100));
    cout << "Correctness checks passed.\n";
}

// ================================================
// Экспериментальные измерения
// ================================================
volatile uint64_t search_checksum = 0;
struct Statistics {
    double mean = 0, m2 = 0;
    unsigned samples = 0;
    void Add(double value) {
        ++samples;
        const double delta = value - mean;
        mean += delta / samples;
        m2 += delta * (value - mean);
    }
    // Выборочное стандартное отклонение вычисляется при записи.
};
void WriteSummary(ostream& out, const string& kind, const string& title,
                  const string& input_case, size_t n, const Statistics& s) {
    const double sd = s.samples > 1 ? sqrt(s.m2 / (s.samples - 1)) : 0;
    out << kind << ',' << title << ',' << input_case << ',' << n << ','
        << s.samples << ',' << s.mean << ',' << sd << '\n';
    cout << title << " n=" << n << ' ' << input_case << ": " << s.mean
         << " ms/operation, sd=" << sd << endl;
}
void TestSort(const string& title, SortFunction func, Order order,
              size_t size, unsigned count, double target_ms,
              ostream& raw, ostream& summary) {
    if (size == 0 || count == 0 || target_ms <= 0) throw invalid_argument("Invalid test
    parameters");
    // Один и тот же исходный массив используется во всех повторениях
    // данного опыта; перед КАЖДОЙ сортировкой он копируется заново.
    generator.seed(42 + static_cast<unsigned>(size));
    const Vector original = MakeVector(order, size);
    Vector expected = original;
    StdSort(expected); // Эталон вне таймера.
    Vector v = original;
    func(v); // Прогрев.
    Require(v == expected);
```

Код программы (продолжение)

```
    Statistics stats;
    const string order_str = order == Rnd ? "random" : order == Asc ? "ascending" : "descending";
    SortFunction volatile measured = func;
    for (unsigned repeat = 1; repeat <= count; ++repeat) {
        double total_ms = 0;
        size_t runs = 0;
        do {
            v = original; // Вне таймера, в том числе для Dsc!
            const auto start = Clock::now();
            measured(v);
            total_ms += duration<double, milli>(Clock::now() - start).count();
            Require(v == expected); // Вне таймера.
            ++runs;
        } while (total_ms < target_ms);
        const double per_operation = total_ms / static_cast<double>(runs);
        stats.Add(per_operation);
        raw << "sort," << title << ',' << order_str << ',' << size << ',' << repeat
            << ',' << runs << ',' << total_ms << ',' << per_operation << '\n';
    }
    WriteSummary(summary, "sort", title, order_str, size, stats);
}
void TestSearch(const string& title, SearchFunction func, size_t size,
                bool existing, unsigned count, double target_ms,
                ostream& raw, ostream& summary) {
    if (size == 0 || count == 0 || target_ms <= 0) throw invalid_argument("Invalid test
    parameters");
    Vector vec = AscendingVector(size);
    uint64_t checksum = 0;
    // Проверка и прогрев вне измерений.
    for (size_t i = 0; i < size; ++i) {
        const T key = existing ? vec[i] : -1000;
        Require(func(vec, key) == (existing ? static_cast<int>(i) : -1));
    }
    Statistics stats;
    // Volatile относится к указателю: вызов остаётся косвенным, что мешает
    // оптимизатору подменить/вынести повторяющийся поиск из цикла.
    SearchFunction volatile measured = func;
    for (unsigned repeat = 1; repeat <= count; ++repeat) {
        double total_ms = 0;
        size_t queries = 0;
        do {
            const auto start = Clock::now();
            // Полный проход по всем ключам, как предложено в задании.
            for (size_t i = 0; i < size; ++i) {
                const T key = existing ? vec[i] : -1000;
                checksum += static_cast<uint64_t>(measured(vec, key) + 1);
            }
            total_ms += duration<double, milli>(Clock::now() - start).count();
            queries += size;
        } while (total_ms < target_ms);
        search_checksum = checksum; // Наблюдаемый результат вычислений.
        const double per_operation = total_ms / static_cast<double>(queries);
        stats.Add(per_operation);
        raw << "search," << title << ',' << (existing ? "found" : "missing")
            << ',' << size << ',' << repeat << ',' << queries << ',' << total_ms
            << ',' << per_operation << '\n';
    }
    WriteSummary(summary, "search", title, existing ? "found" : "missing", size, stats);
}

// ================================================
// Основная программа
// ================================================
int main(int argc, char** argv) {
    try {
        bool quick = false, check_only = false;
        for (int i = 1; i < argc; ++i) {
            const string arg = argv[i];
            if (arg == "--quick") quick = true;
            else if (arg == "--check") check_only = true;
            else if (arg == "--help") {
                cout << "asd-lab [--check | --quick]\n"
```

Код программы (продолжение)

```
                    "Default: full experiment, 10 repetitions, >=100 ms per repetition.\n"
                    "CSV files in the current directory are overwritten.\n";
                return 0;
            } else throw invalid_argument("Unknown argument: " + arg);
        }
        CheckAlgorithms();
        if (check_only) return 0;
        const unsigned count = quick ? 2 : 10;
        const double target_ms = quick ? 1 : 100;
        const size_t low = 250;
        // Пределы не включаются, как в исходном шаблоне преподавателя.
        const size_t quadratic_limit = quick ? 1001 : 10'000;
        const size_t merge_limit = quick ? 1001 : 3'000'000;
        const size_t linear_limit = quick ? 1001 : 10'000;
        const size_t binary_limit = quick ? 1001 : 1'000'000;
        const string prefix = quick ? "quick-" : "";
        ofstream raw(prefix + "raw.csv"), summary(prefix + "summary.csv");
        if (!raw || !summary) throw runtime_error("Cannot open output CSV files");
        raw << setprecision(12) <<
    "kind,algorithm,case,n,repeat,operations,total_ms,ms_per_operation\n";
        summary << setprecision(12) << "kind,algorithm,case,n,repeats,mean_ms,stddev_ms\n";
        cout << "Mode: " << (quick ? "QUICK CHECK" : "FULL EXPERIMENT")
             << ", repeats=" << count << ", target_ms=" << target_ms << ", seed=42\n";
        for (Order order : {Asc, Dsc, Rnd}) {
            for (size_t size = low; size < quadratic_limit; size *= 2) {
                TestSort("InsertionSort", InsertionSort, order, size, count, target_ms, raw,
    summary);
                TestSort("SelectionSort", SelectionSort, order, size, count, target_ms, raw,
    summary);
            }
            for (size_t size = low; size < merge_limit; size *= 2)
                TestSort("MergeSort", MergeSort, order, size, count, target_ms, raw, summary);
        }
        for (bool existing : {true, false}) {
            for (size_t size = low; size < linear_limit; size *= 2)
                TestSearch("LinearSearch", LinearSearch, size, existing, count, target_ms, raw,
    summary);
            for (size_t size = low; size < binary_limit; size *= 2)
                TestSearch("BinarySearch", BinarySearch, size, existing, count, target_ms, raw,
    summary);
        }
        raw.close(); summary.close();
        if (!raw || !summary) throw runtime_error("Failed writing CSV files");
        cout << "Saved: " << prefix << "raw.csv, " << prefix << "summary.csv\n";
    } catch (const exception& e) {
        cerr << "Error: " << e.what() << '\n';
        return 1;
    }
}
```